K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 2

Implementation of Banker's Algorithm - Deadlock Avoidance

Submitted by:

Name: **Sujal Tomar**

Roll Number: 2401201103

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

1. Objective

To implement **Banker's Algorithm** — a deadlock avoidance technique used in operating systems — and determine whether a given system state is **safe or unsafe**, as well as find the **safe sequence** of process execution if one exists.

2. Tools & Technologies Used

Tool	Purpose
Python 3.x	Implementation language
Built-in libraries: os, `multiprocessing`, `time`	Core utilities
Linux / macOS Terminal	Execution environment
GitHub	Version control and submission

3. Key Concepts

Term	Formula / Description
Allocation Matrix	Resources currently allocated to each process
Maximum Matrix	Maximum resources a process may ever request
Need Matrix	Need = Maximum − Allocation
Available Vector	Resources currently free in the system
Work Vector	Working copy of Available; updated during safety check
Safe State	A state where a safe sequence exists for all processes
Safe Sequence	Order in which all processes can execute without deadlock

4. Task 1 — System Input and Data Representation

The system data is collected from the user: number of processes, number of resource types, the Allocation Matrix, the Maximum Matrix, and the Available resources vector. All data is stored and displayed in tabular format.

Input Data Used

Number of Processes: 5 **Number of Resource Types:** 3

Allocation Matrix

Process	R0	R1	R2
P0 P1	0	1	0
P2 P3	2	0	0
P4	3	0	2
	2	1	1
	0	0	2

Maximum Matrix

Process	R0	R1	R2
P0 P1	7	5	3
P2 P3	3	2	2
P4	9	0	2
	2	2	2
	4	3	3

Available Resources: [3, 3, 2]

Screenshot — Input & Matrices Output

```
============================================================
   INPUT SUMMARY
============================================================

────────────────────────────────────────────
  Allocation Matrix
────────────────────────────────────────────

       R0  R1  R2
  P0  [  0   1   0 ]
  P1  [  2   0   0 ]
  P2  [  3   0   2 ]
  P3  [  2   1   1 ]
  P4  [  0   0   2 ]

────────────────────────────────────────────
  Maximum Matrix
────────────────────────────────────────────

       R0  R1  R2
  P0  [  7   5   3 ]
  P1  [  3   2   2 ]
  P2  [  9   0   2 ]
  P3  [  2   2   2 ]
  P4  [  4   3   3 ]

  Available Resources: [3, 3, 2]
```

5. Task 2 — Need Matrix Calculation

The **Need Matrix** represents how many more resources each process may still request before completing. It is derived using the formula:

> **Need[i][j] = Maximum[i][j] − Allocation[i][j]**

Need Matrix

Process	R0	R1	R2
P0 P1	7	4	3
P2 P3	1	2	2
P4	6	0	0
	0	1	1
	4	3	1

Screenshot — Need Matrix Output

```
────────────────────────────────────────
  Need Matrix  (Max - Allocation)
────────────────────────────────────────
       R0  R1  R2
  P0  [  7   4   3 ]
  P1  [  1   2   2 ]
  P2  [  6   0   0 ]
  P3  [  0   1   1 ]
  P4  [  4   3   1 ]
```

6. Task 3 — Banker's Safety Algorithm Implementation

The Safety Algorithm determines whether the system is currently in a **safe state** by simulating resource allocation in a specific order. It works as follows:

1. Initialize **Work** = Available
2. Initialize **Finish[i]** = False for all processes
3. Find a process `i` where: `Finish[i] == False AND Need[i] ≤ Work`
4. Simulate execution: `Work = Work + Allocation[i]`, set `Finish[i] = True`
5. Repeat steps 3–4 until all processes finish or no eligible process is found
6. If all `Finish[i] == True` → **Safe State**, else → **Unsafe State**

Step-by-Step Trace

Step	Process	Need	Work (before)	Work (after)
1 2	P1 P3	[1, 2, 2]	[3, 3, 2] [5, 3,	[5, 3, 2] [7,
3	P4	[0, 1, 1]	2]	4, 3]
4	P0	[4, 3, 1]	[7, 4, 3]	[7, 4, 5]
5	P2	[7, 4, 3]	[7, 4, 5]	[7, 5, 5]
		[6, 0, 0]	[7, 5, 5]	[10, 5, 7]

Screenshot — Safety Algorithm Trace Output

```
=========================================================
   SAFETY ALGORITHM — STEP-BY-STEP TRACE
=========================================================
  Initial Work (Available) : [3, 3, 2]

  Step 1: P1 can proceed
    Need     : [1, 2, 2]
    Work     : [3, 3, 2]  →  [5, 3, 2]

  Step 2: P3 can proceed
    Need     : [0, 1, 1]
    Work     : [5, 3, 2]  →  [7, 4, 3]

  Step 3: P0 can proceed
    Need     : [7, 4, 3]
    Work     : [7, 4, 3]  →  [7, 5, 3]

  Step 4: P2 can proceed
    Need     : [6, 0, 0]
    Work     : [7, 5, 3]  →  [10, 5, 5]

  Step 5: P4 can proceed
    Need     : [4, 3, 1]
    Work     : [10, 5, 5]  →  [10, 5, 7]
```

7. Task 4 — Safe Sequence Determination

Since all processes could be scheduled successfully, the system is in a **safe state** and the following safe sequence was found:

```
P1 → P3 → P4 → P0 → P2
```

This means processes can be executed in the above order such that each process finds enough resources available (after previously completed processes release theirs), and no process ever has to wait indefinitely.

Screenshot — Safe Sequence Output

```
============================================================
   RESULT ANALYSIS
============================================================

  ✔  System is in a SAFE STATE.

  Safe Sequence : P1 → P3 → P0 → P2 → P4
```

---

8. Task 5 — Result Analysis

System State Verdict

Parameter	Value
System State	☑ SAFE
Safe Sequence Found	P1 → P3 → P4 → P0 → P2
Total Processes	5 3
Total Resource Types	

Analysis

- The Safety Algorithm successfully found a valid execution sequence for all 5 processes.
- At each step, the selected process had its **Need ≤ Work**, meaning it could obtain all required resources and complete execution.
- After each process completed, it released its allocated resources back into Work, enabling the next process to proceed.
- Since all `Finish[i]` values became `True`, the system is confirmed to be in a **safe state** — no deadlock can occur under the current allocation.

Comparison with Theoretical Expectations

- The result matches the expected output from the standard textbook example of Banker's Algorithm.
- In theory, a safe state guarantees the system can always find at least one ordering of processes that avoids deadlock — which is exactly what was observed here.
- If Available resources were reduced (e.g., Available = [1, 0, 0]), no safe sequence would exist and the system would be in an **unsafe state**, risking deadlock.

Screenshot — Final Result Output

```
============================================================
   RESULT ANALYSIS
============================================================

  ✔  System is in a SAFE STATE.

  Safe Sequence : P1 → P3 → P0 → P2 → P4

  Explanation
  ────────────
  The Safety Algorithm found a valid sequence in which
  every process can obtain the resources it needs, execute
  to completion, and release its resources — without any
  process waiting indefinitely.  This guarantees that no
  deadlock will occur under the current allocation.


~/Documents/Sem4/Os/code
❯
```

9. GitHub Repository

Link: Github

---

10. Source Code

```
"""
=========================================================================
   Banker's Algorithm for Deadlock Avoidance
   Course    : Operating System Lab (ENCA252)
   Program : BCA (AI & DS) (Research)
=========================================================================
"""

import os

# ──────────────────────────────────────────────────────────────────────
# TASK 1 — System Input and Data Representation #
──────────────────────────────────────────────────────────────────────

def get_system_input():
    """Prompt the user for all system data and return it as a dictionary."""

    print("=" * 55)
    print("        BANKER'S ALGORITHM — DEADLOCK AVOIDANCE SYSTEM")
    print("=" * 55)

    # Number of processes and resource types
    n = int(input("\nEnter number of processes                    : "))
    r = int(input("Enter number of resource types              : "))

    # ── Allocation Matrix ──────────────────────────────────────────────
    print(f"\n[Allocation Matrix] — {n} processes × {r} resources")
    print("Enter row by row (space-separated values):")
    allocation = []
    for i in range(n):
        row = list(map(int, input(f"            P{i} : ").split()))
        allocation.append(row)

    # ── Maximum Matrix ─────────────────────────────────────────────────
    print(f"\n[Maximum Matrix] — {n} processes × {r} resources")
    print("Enter row by row (space-separated values):")
    maximum = []
```

```
    for i in range(n):
        row = list(map(int, input(f"  P{i} : ").split()))
        maximum.append(row)

    # ── Available Resources ──────────────────────────────
    print(f"\n[Available Resources] — {r} values (space-separated):")
    available = list(map(int, input("  Available : ").split()))

    return {
        "n": n, "r": r, "allocation":
        allocation, "maximum":
        maximum, "available":
        available,

    }


def display_matrix(title, matrix, n, r):
    """Pretty-print any matrix with process labels."""
    print(f"\n{'─'*40}")
    print(f"  {title}")
    print(f"{'─'*40}")
    header = "  " + "  ".join([f"R{j}" for j in range(r)])
    print(header)
    for i in range(n):
        row_str = "  ".join(str(v).rjust(2) for v in matrix[i])
        print(f"  P{i}  [ {row_str} ]")


def display_vector(title, vector):
    """Pretty-print a 1-D resource vector."""
    print(f"\n  {title}: {vector}")


# ──────────────────────────────────────────────────────
# TASK 2 — Need Matrix Calculation
# ──────────────────────────────────────────────────────

def calculate_need(maximum, allocation, n, r):
    """
    Compute the Need matrix.
    Formula : Need[i][j] = Maximum[i][j] − Allocation[i][j]
    """
    need = []
    for i in range(n):
        row = [maximum[i][j] - allocation[i][j] for j in range(r)]
```

```
        need.append(row)
    return need


# ────────────────────────────────────────────────────────────────────────
# TASK 3 — Safety Algorithm      &   TASK 4 — Safe Sequence
# ────────────────────────────────────────────────────────────────────────

def safety_algorithm(n, r, allocation, need, available):
    """
    Banker's Safety Algorithm.

    Steps   -----   1.   Work   2.
    Finish[i] ← False
                    ← Available   (copy, so original is untouched)
                                  for all i
    3. Find process i where:
            Finish[i] == False        AND  Need[i] ≤ Work
    4. Work ← Work + Allocation[i];          Finish[i] ← True
    5. Repeat step 3-4 until no such i exists.
    6. If all Finish[i] == True            →   SAFE STATE
       Otherwise                           →   UNSAFE STATE

    Returns ------- (is_safe : bool, safe_sequence :
    list) """



     work    = available[:]                  # Step 1 — working copy of available
resources
    finish = [False] * n                     # Step 2 — no process has finished yet
    safe_sequence = []
    trace_log = []                           # for detailed step-by-step output

    print("\n" + "=" * 55)
    print("       SAFETY ALGORITHM — STEP-BY-STEP TRACE")
    print("=" * 55)
    print(f"      Initial Work (Available) : {work}\n")

    iteration = 0
    while len(safe_sequence) < n:
        found = False

        for i in range(n):
                if finish[i]:
                  continue   # already completed, skip
```

```
            # Step 3 — check if Need[i] ≤ Work (element-wise)
            if all(need[i][j] <= work[j] for j in range(r)):
                # Step 4 — simulate resource allocation and release
                old_work= work[:]
                work = [work[j] + allocation[i][j] for j in range(r)]
                finish[i] = True
                safe_sequence.append(i)
                found = True

                iteration += 1
                print(f"  Step {iteration}: P{i} finish")
                # restart search from P0
                print(f"  Need   :{need[i]}")
                print(f"  Work   :{old_work} → {work}")
                break proceed
        if not found:
            # No eligible process found — deadlock possible
            break

    is_safe = all(finish)
    return is_safe, safe_sequence


# ────────────────────────────────────────────────
# TASK 5 — Result Analysis and Display
# ────────────────────────────────────────────────

def display_results(is_safe, safe_sequence, n):
    """Display the final verdict and safe sequence (Task 4 + 5)."""

    print("=" * 55)
    print("        RESULT ANALYSIS")
    print("=" * 55)

    if is_safe:
        # Task 4 — Safe sequence
        seq_str = " → ".join([f"P{p}" for p in safe_sequence])
        print("\n            System is in a SAFE STATE.")
        print(f"\n        Safe Sequence : {seq_str}")
        print("""
  Explanation ──────────── The Safety Algorithm found a valid
  sequence in which every process can obtain the resources it needs,
  execute
```

```
    to completion, and release its resources — without any
    process waiting indefinitely.            This guarantees that no
    deadlock will occur under the current allocation.
            """)
        else:
            # Identify which processes could not be scheduled
            safe_set           = set(safe_sequence)
            stuck_procs = [f"P{i}" for i in range(n) if i not in safe_set]
            seq_str = " → ".join([f"P{p}" for p in safe_sequence]) if
safe_sequence else "None"

            print("\n       ✘    System is in an UNSAFE STATE.")
            print(f"\n        Partial sequence found : {seq_str}")
            print(f"       Processes that could not proceed : {stuck_procs}")
            print("""
    Explanation ———————— No safe sequence could be found
    that satisfies all

    processes.      The processes listed above are stuck —
    each requires more resources than what is currently
    available, even after simulating all possible orderings.
    This indicates the system is vulnerable to deadlock.
            """)


# ————————————————————————————
# MAIN — Tie everything together
# ————————————————————————————

def main():
        os.system("clear" if os.name == "posix" else "cls")

        # ── Task 1 — Input ————————————————————————————————————
        data = get_system_input()
        n = data["n"]
        r = data["r"]
        allocation = data["allocation"]
        maximum      = data["maximum"]
        available = data["available"]

        # Display input matrices
        print("\n" + "=" * 55)
        print("        INPUT SUMMARY")
        print("=" * 55)
        display_matrix("Allocation Matrix",              allocation, n, r)
        display_matrix("Maximum Matrix",              maximum,      n, r)
```

```
    display_vector("Available Resources", available)

    # ── Task 2 — Need Matrix ──────────────────────────────
    need = calculate_need(maximum, allocation, n, r)
    display_matrix("Need Matrix        (Max − Allocation)", need, n, r)

    # ── Task 3 & 4 — Safety Algorithm + Safe Sequence ────
    is_safe, safe_sequence = safety_algorithm(n, r, allocation, need,
available)

    # ── Task 5 — Result Analysis ──────────────────────────
    display_results(is_safe, safe_sequence, n)


# ── Entry Point ──────────────────────────────────────────
if __name__ == "__main__":
    main()
```